The Web Application Security Playbook

A complete, detailed guide to web application vulnerability classes — from injection to broken access control

From a single unsanitized parameter to a full account takeover, this playbook builds the vulnerability-class intuition and root-cause understanding that web application security work depends on.

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 15, 2026

Preface

Tools like Burp Suite give you the vantage point to test a web application; this book is about what to actually look for once you have it. Every vulnerability class covered here — injection, broken authentication, XSS, SSRF, and the rest — has a specific root cause in how an application handles untrusted input or enforces trust boundaries, and understanding that root cause is what separates pattern-matching a payload from actually understanding why it works.

The organization follows the OWASP Top 10 closely, since it remains the industry's shared vocabulary for vulnerability classes, but each chapter goes deeper than a checklist entry: what the vulnerability actually is at a mechanical level, how to find it, a concrete example, and how it's actually fixed — because testing and defense are two views of the same underlying issue.

Authorization required

Every technique in this book is intended strictly for authorized security testing, bug bounty programs within their disclosed scope, and personal lab/CTF environments. Testing applications you don't own or have explicit written authorization to test is illegal in most jurisdictions.

What this book covers

- HTTP fundamentals relevant to security testing
- Injection: SQL injection, command injection, and related classes
- Broken authentication and session management
- Broken access control, including IDOR
- Cross-Site Scripting (XSS): reflected, stored, and DOM-based
- Cross-Site Request Forgery (CSRF)
- Security misconfiguration
- Insecure deserialization
- Server-Side Request Forgery (SSRF)
- XML External Entities (XXE)
- File upload vulnerabilities
- API security considerations
- Secure development practices and defense in depth

Swastik Sagar

Contents

1

HTTP Fundamentals for Security Testing

1.1 Why HTTP knowledge matters here

Nearly every web vulnerability manifests as an HTTP request or response that does something the application didn't intend — reading raw requests and responses fluently, not just through a browser's rendered view, is the foundation every other chapter in this book builds on.

1.2 Requests and responses

Part	Security relevance
Method	GET vs. POST affects where parameters live and what gets logged/cached
Headers	Cookies, Authorization, Host, and custom headers all carry trust-relevant data
Parameters	Query string, body, and JSON fields are the primary untrusted-input surface
Status code	Reveals server-side logic branches, even without reading the body

1.3 Statelessness and cookies

HTTP itself is stateless — every request is independent, with no built-in memory of prior requests. Session cookies are the mechanism applications bolt on to simulate state, which is exactly why session handling (chapter 3) is such a common source of vulnerabilities: it's a manually engineered workaround for something the protocol doesn't provide natively.

Read the raw traffic, not just the rendered page

A browser's rendered output hides most of what actually matters for security testing — hidden fields, extra response headers, and the exact parameter names sent are only fully visible in the raw request/response, which is exactly what a proxy like Burp Suite exists to surface.

2

Injection

2.1 The core mechanism

Injection vulnerabilities share one root cause: untrusted input is concatenated directly into a command or query, letting an attacker break out of the intended data context and inject their own instructions.

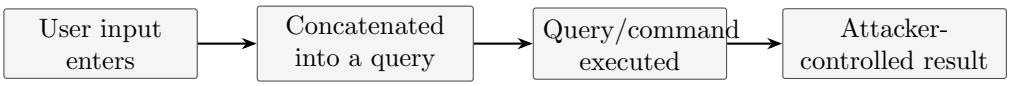

Figure 2.1. *Every injection class – SQL, command, LDAP, and more – follows this same pattern with a different target language.*

2.2 SQL injection

A classic authentication bypass

```
Input:  ’  OR ’1’=’1
Resulting query:
SELECT * FROM users WHERE username=’’ OR ’1’=’1’ AND password=’’
```

The injected `OR ’1’=’1’` makes the WHERE clause always true, returning every row regardless of the actual credentials supplied.

Variant	**Behavior**
In-band (classic)	Results returned directly in the application’s response
Blind (boolean)	No visible data, but true/false conditions change the response subtly
Blind (time-based)	No visible difference at all; timing (e.g. a deliberate delay) confirms injection
Out-of-band	Data exfiltrated via a separate channel, e.g. a DNS lookup the attacker controls

2.3 Command injection

Chaining onto an intended command

```
Intended:  ping <input>
Injected input:  8.8.8.8; cat /etc/passwd
Resulting command:  ping 8.8.8.8; cat /etc/passwd
```

2.4 The fix, for every injection class

- **Parameterized queries / prepared statements** — the data is never concatenated into the query string at all
- **Avoid shelling out to system commands with user input**, use language-native library calls instead
- **Allowlist validation** where the input has a genuinely limited valid set of values
- Least-privilege database accounts, limiting the blast radius even if injection succeeds

Why escaping alone isn't the real fix

Manually escaping special characters is fragile — it's easy to miss an edge case, and different contexts (SQL, shell, LDAP) escape differently. Parameterized queries sidestep the problem entirely by never treating user input as part of the command syntax in the first place.

3

Broken Authentication

3.1 Common authentication weaknesses

Weakness	Impact
No rate limiting on login	Enables credential stuffing and brute force
Predictable session tokens	Session hijacking by guessing a valid token
Session tokens in the URL	Leaked via browser history, referer headers, and server logs
No session invalidation on logout	A stolen token remains valid indefinitely
Weak password reset flow	Predictable reset tokens or user enumeration via response differences

3.2 Session token requirements

A session token needs to be long and generated with a cryptographically secure random source — anything predictable (sequential IDs, timestamps, weak hashing of user data) can potentially be guessed or brute-forced by an attacker without ever touching valid credentials.

3.3 Testing authentication

- Check for rate limiting: does the application lock out or delay after repeated failed attempts?
- Inspect the session token's entropy and format; is it a JWT, opaque string, or something predictable?
- Test whether a session remains valid after logout
- Test the password reset flow for user enumeration (does the response differ for valid vs. invalid accounts?)

User enumeration is a small leak with big consequences

A login or password-reset flow that responds differently for "account doesn't exist" versus "wrong password" lets an attacker build a list of valid accounts to target with credential stuffing — a small information leak that directly enables a much larger attack.

4

Broken Access Control

4.1 The core mechanism

Broken access control happens when an application checks that a user is *authenticated* but fails to properly verify they're *authorized* for the specific resource or action being requested.

4.2 IDOR: Insecure Direct Object References

A textbook IDOR

```
GET /api/invoices/1042 (belongs to the logged-in user)
GET /api/invoices/1043 (belongs to a different user -- returned anyway)
```

If changing the ID alone returns another user's data without any authorization check server-side, that's IDOR — the vulnerability is the missing check, not the predictable ID itself.

4.3 Vertical vs. horizontal access control

Type	**Example**
Horizontal	A regular user accessing another regular user's data (same privilege level)
Vertical	A regular user accessing an admin-only function (different privilege level)

4.4 Testing for broken access control

- Capture a request as User A, then replay it with User B's session but User A's referenced object ID
- Test whether client-side-hidden functionality (like an admin panel link removed from the UI) is still reachable directly by URL
- Test whether an API enforces the same access checks its corresponding UI does — APIs are frequently checked less rigorously

The fix is always server-side

Access control enforced only in the UI (hiding a button, disabling a menu item) is not a control at all — every authorization decision has to be re-verified server-side on every request, since a client can always be modified or bypassed entirely.

5

Cross-Site Scripting (XSS)

5.1 The core mechanism

XSS happens when untrusted input is rendered back into a page as HTML or JavaScript without proper encoding, letting an attacker's script execute in the context of a victim's browser session — with full access to whatever that session can do.

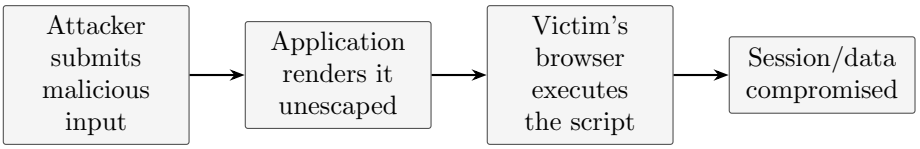

Figure 5.1. *The victim's own browser executes the script with the victim's own privileges – this is what makes XSS dangerous even though the attacker never directly touches the victim.*

5.2 The three XSS types

Type	Mechanism
Reflected	Payload is part of the request and immediately reflected in the response, not stored
Stored	Payload is saved server-side (e.g. a comment) and served to every subsequent visitor
DOM-based	Vulnerability is entirely client-side; JavaScript writes untrusted data into the DOM unsafely

A basic test payload

```
<script>alert(document.domain)</script>
```

Stored XSS is the highest-impact variant

Reflected XSS needs a victim to click a crafted link; stored XSS executes automatically against every user who simply views the affected page — including, in the worst cases, administrators viewing user-submitted content in an admin panel.

5.3 The fix

- Context-aware output encoding: HTML-encode for HTML context, JS-encode for script context, and so on
- Content Security Policy (CSP) as a defense-in-depth layer, restricting what scripts can execute at all

- Modern frameworks (React, Angular, Vue) auto-escape by default — most real-world XSS now comes from deliberately bypassing that default (e.g. `dangerouslySetInnerHTML`)

6

Cross-Site Request Forgery (CSRF)

6.1 The core mechanism

CSRF tricks a victim's already-authenticated browser into submitting a request the victim never intended — the browser automatically attaches the victim's session cookie to any request to that domain, regardless of which page actually triggered it.

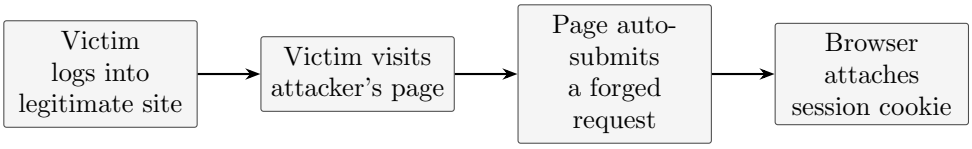

Figure 6.1. *The victim never has to click anything on the attacker's page – an auto-submitting form or image tag is enough to fire the forged request.*

A minimal CSRF proof-of-concept

```
<form action="https://bank.example/transfer" method="POST">
<input type="hidden" name="amount" value="1000">
<input type="hidden" name="to" value="attacker-account">
</form>
<script>document.forms[0].submit()</script>
```

6.2 The fix

- Anti-CSRF tokens: a unique, unpredictable value required on every state-changing request, tied to the user's session
- SameSite cookie attribute (`Strict` or `Lax`), which stops the browser from attaching cookies to most cross-site requests entirely
- Re-verifying sensitive actions with a fresh credential check (e.g. re-entering a password before a critical change)

SameSite changed the landscape significantly

Modern browsers defaulting cookies to `SameSite=Lax` has closed a large fraction of classic CSRF on its own — but it isn't a complete substitute for anti-CSRF tokens, particularly for applications that need `SameSite=None` for legitimate cross-site use cases.

7

Security Misconfiguration

7.1 Common misconfigurations

Misconfiguration	Risk
Default credentials left unchanged	Immediate unauthorized access
Verbose error messages in production	Leaks stack traces, file paths, framework versions
Directory listing enabled	Exposes files never meant to be browsable
Unnecessary services/ports exposed	Expands attack surface beyond what's actually needed
Missing security headers	Removes defense-in-depth layers (CSP, HSTS, X-Frame-Options)

7.2 Key security headers

Header	Purpose
Content-Security-Policy	Restricts what scripts/resources a page can load or execute
Strict-Transport-Security	Forces HTTPS for future requests to the domain
X-Frame-Options	Prevents the page from being embedded in a clickjacking iframe
X-Content-Type-Options	Prevents the browser from MIME-sniffing away a declared content type

Misconfiguration is often the easiest finding

Unlike injection or logic flaws, misconfiguration issues are usually detectable with a single request and no crafted payload at all — which is exactly why passive scanning (Burp's passive scanner, for instance) catches this category so reliably.

8

Insecure Deserialization

8.1 The core mechanism

Many languages support serializing objects into a byte stream for storage or transport, then deserializing them back into live objects — if an application deserializes attacker-controlled data without validation, a crafted serialized payload can trigger unintended code execution during the deserialization process itself.

Why this is more dangerous than it sounds

Deserialization vulnerabilities often lead directly to remote code execution, not just data corruption — because the deserialization process itself can be made to instantiate objects and call methods chosen entirely by the attacker, before the application ever gets to apply its own logic to the result.

8.2 Where it shows up

- Session state stored in a serialized cookie or hidden field
- Inter-service communication using a native serialization format
- Caching layers storing serialized objects

8.3 The fix

- Avoid deserializing untrusted data entirely where possible; prefer plain data formats like JSON with strict schema validation
- If native deserialization is unavoidable, use integrity checks (signing) to detect tampering before deserializing
- Run deserialization in a sandboxed, least-privilege context

9

Server-Side Request Forgery (SSRF)

9.1 The core mechanism

SSRF occurs when an application fetches a URL supplied by the user without properly restricting the destination — letting an attacker make the server itself issue requests to internal systems it wasn't meant to expose.

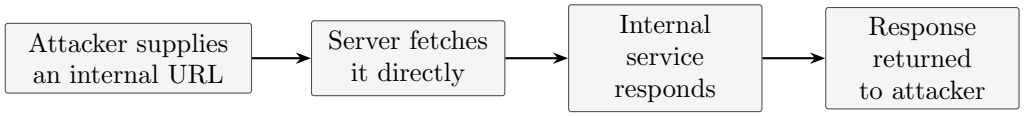

Figure 9.1. *The server acts as a proxy into its own internal network on the attacker's behalf – this is what makes SSRF valuable even without any direct network access.*

A classic cloud-metadata SSRF target

```
http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

On many cloud platforms, this internal-only address exposes instance credentials — a frequent, high-impact SSRF target specifically because it can lead directly to broader cloud account compromise.

9.2 The fix

- Allowlist permitted destination hosts/protocols rather than trying to blocklist dangerous ones
- Block requests to private IP ranges and the cloud metadata address at the network layer, not just application logic
- Disable unnecessary URL schemes (like `file://` or `gopher://`) in whatever library performs the fetch

10

XML External Entities (XXE)

10.1 The core mechanism

XML parsers can be configured to resolve external entities referenced within a document — if an application parses attacker-supplied XML with this feature enabled, a crafted entity definition can read local files or trigger SSRF-style requests from the parser itself.

A classic XXE payload reading a local file

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<foo>&xxe;</foo>
```

10.2 The fix

- Disable external entity resolution (and DTD processing generally) in the XML parser's configuration — this is a one-line fix in most modern XML libraries
- Prefer JSON over XML for data interchange where the format choice is still open
- Validate and sanitize any XML input even after disabling external entities, as defense in depth

Often a default configuration issue

Many XML libraries historically shipped with external entity resolution enabled by default — meaning XXE frequently isn't a coding mistake so much as an unpatched or unconfigured library default, making it worth checking explicitly rather than assuming a modern library is safe out of the box.

11

File Upload Vulnerabilities

11.1 Common file upload weaknesses

Weakness	Risk
No file type validation	Uploading an executable script disguised as a document
Client-side-only validation	Trivially bypassed by intercepting and modifying the request
Predictable upload paths	Attacker can locate and directly request an uploaded malicious file
Uploaded files served from the same origin	An uploaded HTML/SVG file can carry a stored XSS payload

11.2 Bypassing weak validation

- Testing double extensions (`shell.php.jpg`) against extension-based filters
- Modifying the `Content-Type` header directly, since many checks trust it without verifying actual content
- Embedding a payload within a valid file structure (polyglot files) to pass content-based checks

11.3 The fix

- Validate file type by content (magic bytes), not filename extension or client-supplied Content-Type
- Store uploads outside the webroot, or serve them from a separate domain with no execution permissions
- Rename uploaded files to a generated, unpredictable name rather than trusting the original filename

12

API Security

12.1 How API risks differ from traditional web apps

APIs expose the same underlying vulnerability classes as any web application, but often with less client-side logic obscuring the raw endpoints — and are frequently tested less rigorously than the UI that sits in front of them, since access control and validation are sometimes assumed to be "the UI's problem."

12.2 OWASP API Security highlights

Risk	Description
Broken object-level authorization	The API equivalent of IDOR, often the single most common API finding
Excessive data exposure	Returning a full object when the client only needed a few fields, relying on the client to filter
Lack of rate limiting	Enables brute force, scraping, and resource exhaustion
Mass assignment	Accepting and applying client-supplied fields the API never intended to expose for writing

Test the API directly, not just through the UI

An API often accepts requests the front-end UI never actually sends — testing only through the rendered application misses entire parameters and endpoints that a direct API client (or an attacker reading the JavaScript bundle) can reach easily.

13

Secure Development and Defense in Depth

13.1 Shifting left

Finding vulnerabilities in production is the most expensive place to find them — secure coding standards, code review with security in mind, and automated static/dynamic analysis integrated into the development pipeline all catch issues earlier and cheaper than a post-deployment penetration test.

13.2 Defense in depth, applied to web applications

- Input validation at the boundary, output encoding at every rendering context
- Least-privilege database and service accounts, limiting what a successful injection can actually reach
- Security headers and CSP as a backstop even if an XSS filter is somehow bypassed
- WAF rules as a detection/mitigation layer, never a substitute for fixing the underlying code
- Centralized logging feeding a SIEM, so exploitation attempts are visible even when they don't succeed

No single layer is sufficient alone

Every vulnerability class in this book has a specific root-cause fix, but real applications benefit from overlapping defenses — so that a single missed edge case in one layer doesn't translate directly into a full compromise.